

[logo TSP]
images/logo-tsp.png

TÉLÉCOM SUDPARIS — CYCLE INGÉNIEUR

MODULE PRO 3600 — PROJET DE DÉVELOPPEMENT LOGICIEL

RAPPORT DE PROJET

Plateforme web de backtesting algorithmique

Équipe projet :

Adrien ETIENNE Neil KIUCHI Elio BONNENFANT
Robinson CERCEAU Mattéo MACHNIK-HAUCHART

Encadrant du projet :

M. ROMERO

Version du document : 3.0 (Livrable final : logiciel, tests et rapport)

Date : 1^{er} juin 2026

Table des matières

1	Introduction	2
2	Cadrage du projet et périmètre	2
2.1	Motivation et objectifs	2
2.2	Définition du Produit Minimum Viable (MVP)	2
2.3	Évolution du périmètre : au-delà du MVP	3
2.4	Stack technologique	3
3	Organisation de l'équipe	3
3.1	Attribution des rôles et pôles	3
3.2	Méthodologie de travail et outils collaboratifs	3
4	Gestion et histoire du projet	4
4.1	Planification globale et jalons	4
4.2	Stratégie de découpage en sprints	5
4.3	Choix technologiques et justifications	5
4.4	Problèmes rencontrés et solutions	5
5	Conception détaillée	6
5.1	Architecture générale	6
5.2	Backend : structure et API	7
5.3	Frontend : structure et logique	9
5.4	Flux de bout en bout d'un backtest	9
6	Réalisation : fonctionnalités livrées	9
7	Tests et validation	11
7.1	Résultat	11
8	Bilan : objectif vs réalisation	12
8.1	Comparaison au périmètre MVP	12
8.2	Limitations connues et dette technique	12
8.3	Perspectives	12
9	Manuel utilisateur	12
9.1	Prérequis	12
9.2	Installation et lancement	13
9.3	Utilisation pas à pas	13
9.4	Exécution des tests (développeurs)	13
10	Conclusion	14
11	Références	14
12	Lexique	14

1 Introduction

Ce document constitue le rapport final du projet PRO3600 mené par notre équipe. Il prolonge et enrichit le pré-rapport (version 1.0) en y intégrant l'histoire complète du projet, la conception détaillée du logiciel effectivement livré, les résultats de la campagne de tests, un bilan objectif/-réalisation et un manuel utilisateur. Conformément aux attendus du livrable 3, il vise à permettre à une personne ignorant tout du projet d'en comprendre les tenants et aboutissants, à un développeur de maintenir le logiciel, et à un utilisateur de se servir de l'application.

Le logiciel livré est une **plateforme web de backtesting de stratégies de trading** sur données financières réelles. Elle est composée d'une API **FastAPI** (Python) et d'une application monopage **Angular 21**, et permet à un utilisateur de tester des stratégies d'analyse technique (MACD, RSI, croisement de moyennes mobiles) sur des indices boursiers, d'en visualiser les performances et de conserver un historique de ses analyses.

2 Cadrage du projet et périmètre

2.1 Motivation et objectifs

En finance quantitative, il est indispensable de tester rigoureusement une stratégie d'investissement sur des données historiques avant de l'appliquer sur les marchés réels avec de véritables capitaux. Ce processus de simulation, appelé *backtesting*, permet de valider des intuitions mathématiques et d'évaluer la robustesse d'un modèle face aux conditions passées. L'objectif du projet est de concevoir une plateforme web interactive dédiée à cette pratique, agissant comme un laboratoire virtuel d'analyse de modèles d'investissement.

Les solutions de backtesting existantes sont souvent complexes ou nécessitent des compétences avancées en programmation (scripts, API). La valeur ajoutée de notre projet est d'offrir une alternative **ergonomique et visuelle**, rendant l'analyse quantitative accessible directement depuis un navigateur web : l'application évalue automatiquement performances et risques, et restitue les résultats sous forme de graphiques et de métriques financières standards.

2.2 Définition du Produit Minimum Viable (MVP)

L'objectif du premier livrable était de valider l'architecture globale (communication Front-End / Back-End) et de démontrer la faisabilité des calculs. Le MVP visait à permettre à un utilisateur de charger un jeu de données, d'y appliquer une stratégie de base et de visualiser le résultat financier. Le périmètre du MVP comprenait :

- 1) **Gestion des données** : chargement d'un historique financier local (CSV, p. ex. le S&P 500 sur 5 ans).
- 2) **Moteur de backtesting** : implémentation de plusieurs stratégies d'analyse technique standard (MA, MA Crossover, MACD, RSI) et simulation de l'exécution des ordres d'achat/-vente.
- 3) **Évaluation et métriques** : calcul du rendement cumulé de la stratégie sur la période.
- 4) **Interface utilisateur** : une interface web, un bouton de déclenchement du backtest, un graphique du prix de l'actif et l'affichage de la performance finale.

Étaient explicitement *exclus* du MVP : la connexion à des API publiques temps réel, une base de données pour les historiques utilisateurs, des stratégies complexes multiples et les métriques de risque avancées (Sharpe, volatilité).

2.3 Évolution du périmètre : au-delà du MVP

Une fois le MVP validé, l'équipe a poursuivi le développement bien au-delà du périmètre initial. Plusieurs fonctionnalités classées *à après-MVP* dans le pré-rapport ont en réalité été réalisées :

- connexion à une **API publique temps réel** (Yahoo Finance via `yfinance`) à la place du CSV statique ;
- **base de données** (Supabase PostgreSQL) pour persister l'historique des backtests par utilisateur ;
- **métriques de risque avancées** : ratio de Sharpe annualisé, *maximum drawdown*, taux de réussite, nombre de transactions ;
- **authentification** via Google OAuth et un mode démo sans compte.

Ce dépassement du périmètre est analysé dans le bilan (section 8).

2.4 Stack technologique

Couche	Technologies
Back-End	Python 3.11, FastAPI, Pandas, NumPy, <code>yfinance</code> , SQLAlchemy 2.0
Base de données	SQLite (développement) / PostgreSQL Supabase (production)
Authentification	Authlib (Google OAuth/OIDC), sessions signées Starlette
Front-End	Angular 21 (standalone), Angular Material, <code>lightweight-charts</code>
Tests	pytest + coverage (back) ; Vitest + Angular TestBed (front)
Outils	Git & GitHub, Discord, Google Drive

3 Organisation de l'équipe

Afin de paralléliser le travail et de garantir l'efficacité du développement, l'équipe a défini des pôles de compétences (non figés).

3.1 Attribution des rôles et pôles

Pôle	Responsables	Périmètre
Frontend	Robinson Cerceau, Elio Bonnenfant	SPA Angular, interface de paramétrage, intégration des graphiques, appels API.
Backend (architecture & API)	Neil Kiuchi, Mattéo Machnik-Hauchart	Architecture serveur, endpoints, gestion des requêtes, sécurisation et erreurs.
Data	Elio Bonnenfant	Extraction/nettoyage des historiques de prix, format OHLCV, intégration de l'API publique.
Backtesting & stratégies	Adrien Etienne	Simulateur de trading, algorithmes d'analyse technique, génération des signaux, métriques d'évaluation.

3.2 Méthodologie de travail et outils collaboratifs

Nous avons adopté une approche **Agile** : le développement est découpé en cycles itératifs (sprints), permettant de rester flexibles, d'adapter les objectifs aux difficultés rencontrées et de

livrer de manière incrémentale. Un **diagramme de Gantt** complète cette approche pour une vision macroscopique et le repérage des jalons (livrables).

L’environnement collaboratif s’articule autour de trois outils :

- **Discord** — communication temps réel (salons thématiques, points d’avancement, réunions vocales) ;
- **GitHub** — gestion de version du code, suivi des modifications, travail simultané par branches (**features**, **fix**) ;
- **Google Drive** — centralisation documentaire (cahiers des charges, comptes rendus, recherches bibliographiques, rédaction des rapports).

4 Gestion et histoire du projet

4.1 Planification globale et jalons

Un plan de charge sous forme de tableur a recensé l’ensemble des tâches, estimé les temps de réalisation et réparti les responsabilités ; mis à jour régulièrement, il a permis de comparer estimations et temps réellement investi. Le diagramme de Gantt prévisionnel a servi de feuille de route. Les jalons principaux sont rappelés ci-dessous.

Jalon	Échéance	Contenu
Livable 1	23 février	Définition du MVP, planning du sprint 1
Livable 2	27 mars	Itérations intermédiaires
Livable 3	15 mai	Logiciel, tests et rapport final
Soutenance	11–12 juin	Présentation et démonstration

Phases planifiées (Gantt) : choix de l’architecture (janv.–23 fév.), choix back/front (15 janv.–23 fév.), MVP (22 fév.–8 mars), développement frontend (22 fév.–15 mai), développement backend avec données temps réel (22 fév.–24 avr.), modélisation des tables de la base (3 mars–7 mai), développement du moteur de backtesting et stratégies (10 mars–7 mai), phase de test (10 avr.–15 mai), préparation de la soutenance (15 mai–11 juin).



FIGURE 1 – Diagramme de Gantt prévisionnel du projet (jalons et phases de développement).

4.2 Stratégie de découpage en sprints

Conformément à l'approche Agile, le développement a été découpé en itérations courtes dotées d'objectifs précis. Le **sprint 1** (MVP, échéance 9 mars) visait à mettre en place l'environnement de travail et à valider la communication de bout en bout entre Angular et FastAPI sur un jeu de données.

Carnet du sprint 1 (extrait).

- **Pôle Architecture** : création du dépôt GitHub, arborescence `/backend` et `/frontend`, `.gitignore`, README de lancement.
- **Pôle Back-End & Data** : initialisation de FastAPI et d'une route de test, lecture d'un CSV avec Pandas, route renvoyant les données en JSON.
- **Pôle Front-End** : initialisation du projet Angular, structure des pages, intégration d'une librairie de graphiques, appel `fetch` vers le backend.

Les sprints suivants ont porté sur le remplacement du CSV par l'API temps réel, la modélisation de la base de données, le moteur de backtesting et les stratégies, l'authentification, puis la phase de tests et de stabilisation.

4.3 Choix technologiques et justifications

- **FastAPI** : framework Python moderne, performant et auto-documenté (OpenAPI/Swagger), idéal pour exposer rapidement une API typée avec validation des données (Pydantic).
- **Pandas / NumPy** : manipulation vectorisée des séries temporelles financières, calculs d'indicateurs et de rendements concis et efficaces.
- **yfinance** : accès gratuit aux données de marché Yahoo Finance, retenu pour passer du CSV statique à des données réelles sans coût d'API.
- **Angular 21 (standalone)** : framework structurant, typé (TypeScript), avec un système de *signals* pour la réactivité et Angular Material pour une UI cohérente.
- **lightweight-charts** (TradingView) : librairie de graphiques financiers performante (chandelières, courbes) adaptée aux séries longues.
- **SQLAlchemy 2.0 + PostgreSQL (Supabase)** : ORM moderne et base managée pour persister les historiques ; SQLite en développement local.

4.4 Problèmes rencontrés et solutions

- **Colonnes MultiIndex de yfinance** : pour un seul ticker, `yfinance` renvoie des colonnes hiérarchiques. Solution : aplatissement via `df.columns.get_level_values(0)` et renommage de l'index en `Time`.
- **Biais de look-ahead** : un backtest naïf utiliserait le signal du jour t pour le rendement du jour t , ce qui n'est pas autorisé. Solution : décalage du signal d'une barre (`signal.shift(1)`) dans le moteur, de sorte que la position décidée en t s'applique au rendement de $t+1$.
- **CORS Front/Back** : l'application Angular (port 4200 / Vercel) appelle l'API (port 8000 / Render) depuis une autre origine. Solution : configuration du `CORSMiddleware` avec la liste blanche des origines.
- **Authentification Google** : mise en place du flux OAuth/OIDC (Authlib) avec redirection vers le frontend après connexion.
- **Angular 21 n'est pas zoneless** : la nouvelle version n'utilise plus `zone.js`, ce qui a nécessité d'adapter la réactivité (`signals`, `effect`) et la configuration des tests.
- **Outillage de test** : le *scaffolding* de tests frontend n'était pas câblé. Solution : ajout d'un `tsconfig.spec.json` incluant les `*.spec.ts` et des types `vitest/globals` (voir section 7).

5 Conception détaillée

5.1 Architecture générale

La plateforme repose sur une architecture client/serveur découplée, organisée en trois couches aux responsabilités distinctes :

Couche présentation (client).

Une application monopage (SPA) Angular 21, déployée sur Vercel. Elle gère l’affichage et les interactions : les pages (connexion, tableau de bord, données, backtests, mode avancé, résultats, paramètres), les services d’accès aux données et à l’authentification, et les composants de graphiques financiers.

Couche application (serveur).

Une API REST FastAPI, déployée sur Render. Elle reçoit les requêtes du client, récupère les cours de marché, exécute les stratégies et le moteur de backtest, calcule les métriques et gère l’authentification.

Couche persistance (données).

Une base de données relationnelle accédée via l’ORM SQLAlchemy 2.0 : SQLite en développement local, PostgreSQL (hébergé sur Supabase) en production.

Les couches communiquent par des contrats clairs. Le client appelle l’API en **HTTP/JSON** (style REST). Le serveur s’appuie sur deux services externes : **Yahoo Finance** (via la librairie `yfinance`) pour les cours, avec une **mise en cache** de 24 heures en base afin de limiter les appels réseau, et **Google** (OAuth 2.0 / OIDC) pour l’authentification. La persistance des utilisateurs, des backtests et du cache passe par SQLAlchemy. La figure 2 en donne une vue d’ensemble, et la figure 3 détaille le schéma de la base de données.

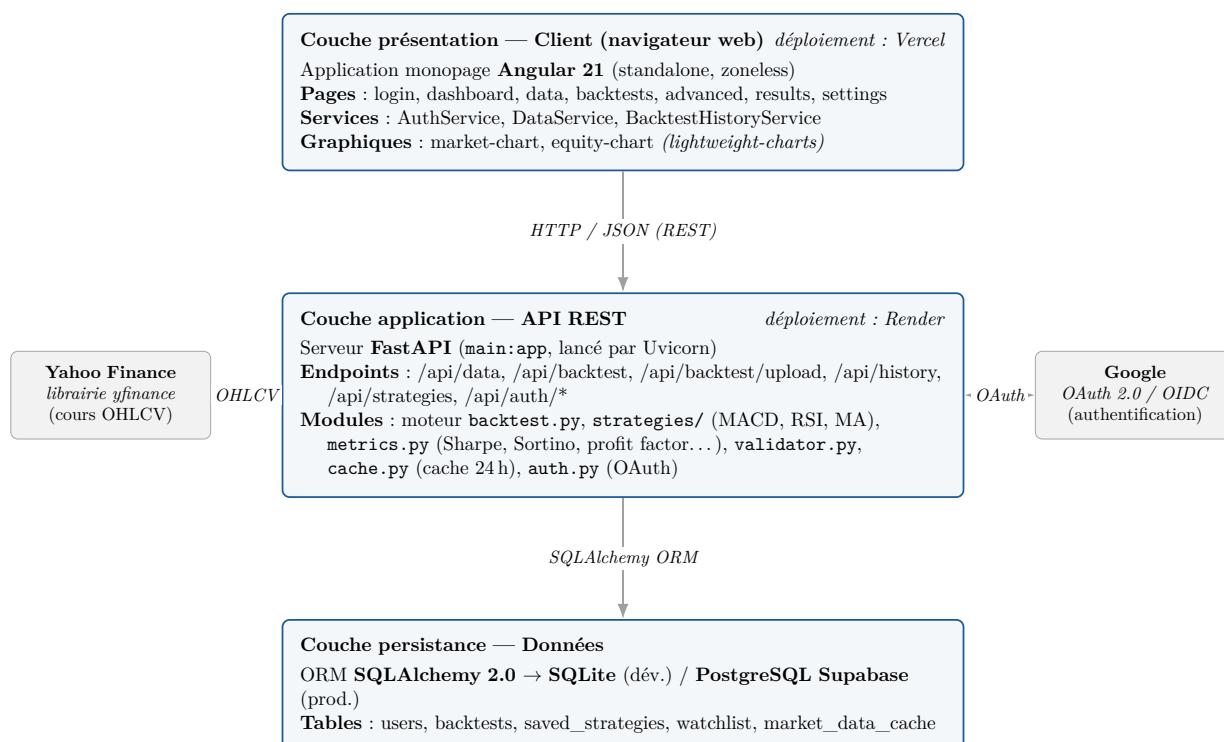


FIGURE 2 – Architecture globale de la plateforme : trois couches découplées (présentation Angular, application FastAPI, persistance SQLAlchemy) et deux services externes (Yahoo Finance pour les cours, Google pour l’authentification).

5.2 Backend : structure et API

Le backend est organisé en modules à responsabilité unique :

Fichier	Rôle
<code>main.py</code>	Point d'entrée, middlewares, endpoints backtest/historique/stratégies/upload
<code>data.py</code>	Routeur <code>/api/data</code> : récupération OHLCV pour les graphiques
<code>backtest.py</code>	Moteur de backtest vectorisé (rendements, métriques)
<code>strategies/</code>	Stratégies (MACD, RSI, MA crossover) + registre
<code>metrics.py</code>	Métriques du <i>mode avancé</i> (Sortino, profit factor, espérance...) sur trades importés
<code>validator.py</code>	Validation des backtests importés (mode avancé)
<code>cache.py</code>	Cache des cours de marché en base (TTL 24 h)
<code>models.py</code>	Modèles ORM (User, Backtest, SavedStrategy, Watchlist, MarketDataCache)
<code>database.py</code>	Configuration SQLAlchemy (engine, session, <code>get_db</code>)
<code>auth.py</code>	Routeur <code>/api/auth</code> : Google OAuth, gestion de compte

Endpoints de l'API.

Méthode	Route	Description
GET	<code>/api/data/{ticker}?period=</code>	Données OHLCV (yfinance)
POST	<code>/api/backtest</code>	Lance un backtest
POST	<code>/api/backtest/upload</code>	Backtest en mode avancé (import de trades JSON)
GET	<code>/api/history/{user_id}</code>	Historique d'un utilisateur
DELETE	<code>/api/history/{id}</code>	Supprime un backtest
GET	<code>/api/strategies</code>	Liste des stratégies
GET	<code>/api/auth/login</code>	Redirection vers Google
GET	<code>/api/auth/callback</code>	Retour OAuth, création/MAJ utilisateur
PATCH	<code>/api/auth/users/{id}</code>	Renommage d'utilisateur
DELETE	<code>/api/auth/users/{id}</code>	Suppression de compte

Modèle de données. La base comporte cinq tables (figure 3). La table `users` et la table `backtests` constituent le cœur persistant : un enregistrement de backtest référence son utilisateur (clé étrangère `user_id`) et stocke le contexte (ticker, stratégie, période, paramètres, capital, stop-loss) ainsi que les métriques calculées (rendement stratégie/marché, Sharpe, drawdown, nombre de transactions, taux de réussite). La table `market_data_cache` met en cache les cours récupérés auprès de Yahoo Finance, avec une contrainte d'unicité sur le couple (`ticker`, `period`) et une durée de vie de 24 heures. Deux tables (`saved_strategies`, `watchlist`) sont préparées pour des évolutions.

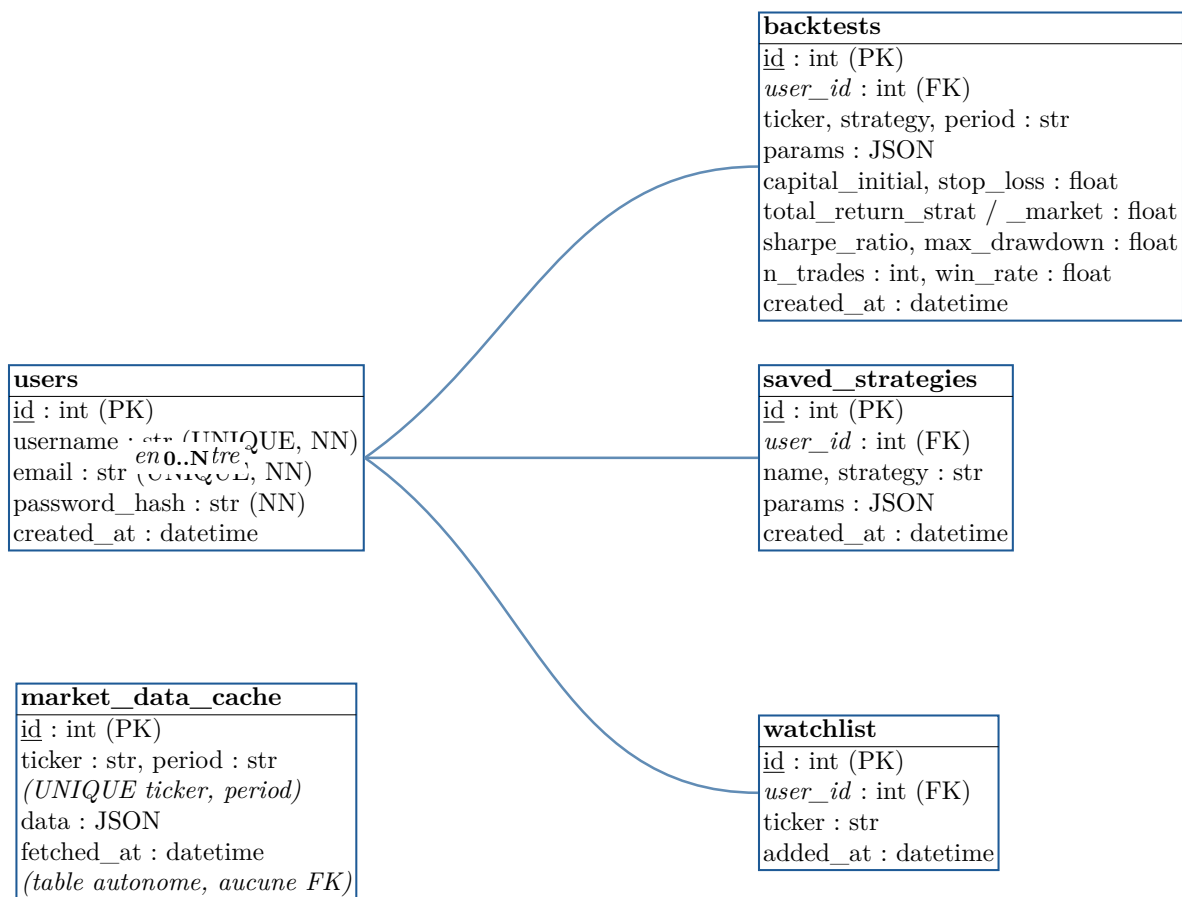


FIGURE 3 – Schéma relationnel de la base de données (cardinalités 1 :N). Un **users** possède 0..N lignes dans **backtests**, **saved_strategies** et **watchlist** via la clé étrangère `user_id`; réciproquement, chaque ligne appartient à exactement un utilisateur. **market_data_cache** est autonome (cache des cours, contrainte d'unicité sur (`ticker`, `period`)). *PK* : clé primaire • *FK* : clé étrangère • *NN* : non nul.

Moteur de backtest. Le moteur (`backtest.py`) est entièrement vectorisé. À partir d'un DataFrame contenant la colonne de signaux, il calcule :

```

1 market_return = Close.pct_change()
2 strat_return  = market_return * signal.shift(1)    # decalage = anti look-ahead
3 cumulative    = (1 + return).cumprod() - 1

```

Les métriques produites par le moteur standard sont : rendement cumulé stratégie et marché, ratio de Sharpe annualisé ($\sqrt{252}$), *maximum drawdown*, nombre de transactions (changements de signal) et taux de réussite (proportion de barres en position gagnantes). Le *mode avancé* (import de trades, calculé par le module `metrics.py`) produit en complément des métriques supplémentaires : ratio de **Sortino**, **profit factor**, espérance par trade, volatilité annualisée, etc.

Stratégies. Toutes les stratégies héritent d'une interface commune **BaseStrategy** et implémentent `compute_signals(df) -> df`, qui ajoute une colonne **signal** (+1 long, -1 court, 0 neutre) et des colonnes d'indicateurs. Un registre (**STRATEGIES_REGISTRY**) associe une clé textuelle à chaque classe, consommée par la route `/api/backtest`.

Stratégie	Paramètres	Logique de signal
MACD	fast=12, slow=26, signal=9	+1 si MACD_line > signal, sinon -1
RSI	length=14, ob=70, os=30	+1 si RSI < os, -1 si RSI > ob, sinon 0
MA crossover	fast=10, slow=30	+1 si MA_fast > MA_slow, sinon -1
Bollinger	length=20, num_std=2	+1 sous la bande basse, -1 au-dessus de la bande haute, sinon 0
Momentum	length=10	+1 si variation sur N barres > 0, -1 si < 0
Donchian	length=20	+1/-1 à la cassure du plus haut/bas des N barres précédentes

5.3 Frontend : structure et logique

L'application Angular est *standalone* (sans NgModule). Le composant racine affiche une barre de navigation (navbar) qui héberge le <router-outlet>. Les pages sont chargées paresseusement (lazy loading) et protégées par un garde de route.

Router. login et callback sont publiques ; les pages dashboard, data, backtests, results, settings sont protégées par authGuard.

Services.

- **DataService** — tous les appels REST (données, backtest, historique, compte) ; l'URL de base provient de environment.apiUrl.
- **AuthService** — identité stockée en localStorage, mode démo, état d'authentification.
- **BacktestHistoryService** — historique local (mode démo), tampon circulaire des 5 derniers résultats.

Composants graphiques. market-chart affiche les cours (chandeliers ou ligne selon la période) et equity-chart la courbe d'équité (stratégie vs marché, rebasée en base 100), via lightweight-charts.

5.4 Flux de bout en bout d'un backtest

1. L'utilisateur choisit ticker, période, stratégie et paramètres sur la page Backtests et clique sur **Lancer**.
2. DataService.runBacktest() envoie un POST /api/backtest.
3. L'API valide la stratégie, récupère les données, instancie la stratégie, calcule les signaux puis exécute le moteur de backtest.
4. La réponse (stats, equity_curve, signals) est renvoyée ; si l'utilisateur est connecté, le résultat est persisté.
5. Le frontend ajoute le résultat à l'historique local et navigue vers la page Results qui affiche métriques, courbe d'équité et tableau de signaux.

6 Réalisation : fonctionnalités livrées

- Récupération de données de marché temps réel (S&P 500, NASDAQ, NQ) sur plusieurs périodes (1J, 1M, 1A, 5A) et visualisation graphique.

- Lancement de backtests sur six stratégies paramétrables (MACD, RSI, MA crossover, Bollinger, Momentum, Donchian) avec calcul de métriques.
- Page de résultats : métriques, courbe d'équité interactive, tableau des signaux, navigation dans l'historique récent.
- Tableau de bord synthétique (nombre de backtests, meilleur rendement, runs récents).
- Authentification Google OAuth + mode démo ; persistance de l'historique en base pour les utilisateurs connectés.
- Page de paramètres (nom d'utilisateur, préférences d'interface, suppression de compte).

Les captures ci-dessous illustrent les principales interfaces livrées.



FIGURE 4 – Tableau de bord : synthèse des backtests et meilleur rendement.



FIGURE 5 – Page Backtests : sélection de l'indice, de la période, de la stratégie et de ses paramètres.



FIGURE 6 – Page Résultats : métriques, courbe d'équité (stratégie vs marché) et tableau des signaux.



FIGURE 7 – Page Données de marché : graphique des cours (chandeliers / ligne) via lightweight-charts.

7 Tests et validation

Nous avons écrit deux types de tests automatisés, classiques en développement logiciel :

- des **tests unitaires**, qui vérifient une fonction à la fois (par exemple : une stratégie produit bien un signal cohérent, ou un calcul de rendement donne le bon résultat) ;
- des **tests d'intégration**, qui vérifient que l'API répond correctement de bout en bout (lancer un backtest, lire ou supprimer un historique, et renvoyer une erreur quand la requête est invalide).

Ils sont écrits avec **pytest** côté backend (Python) et avec **Vitest** côté frontend (Angular). Nous avons complété par quelques essais manuels, en utilisant l'application comme un utilisateur (connexion, lancement d'un backtest, lecture des résultats).

7.1 Résultat

L'ensemble des tests passe : 99 tests automatisés sont au vert.

Périmètre	Nombre de tests	Résultat
Backend (pytest)	58	tous au vert
Frontend (Vitest)	41	tous au vert
Total	99	100% au vert

8 Bilan : objectif vs réalisation

8.1 Comparaison au périmètre MVP

Objectif	Prévu (MVP)	Réalisé
Chargement de données financières	Oui (CSV)	Oui (temps réel)
Stratégies d'analyse technique	Oui	Oui (MACD, RSI, MA)
Simulation des ordres / signaux	Oui	Oui
Rendement cumulé	Oui	Oui
Interface web + graphique + performance	Oui	Oui (multi-pages)
API publique temps réel (yfinance)	Après-MVP	Réalisé
Base de données / historique utilisateur	Après-MVP	Réalisé
Métriques de risque avancées (Sharpe, draw-down)	Après-MVP	Réalisé
Authentification (Google OAuth)	Non prévu	Réalisé

Le projet a donc non seulement atteint les objectifs du MVP, mais a aussi livré l'essentiel des fonctionnalités à après-MVP envisagées, ce qui constitue un résultat très satisfaisant au regard du périmètre initial.

8.2 Limitations connues et dette technique

Par souci de transparence, nous recensons les limites identifiées, pistes d'amélioration pour la suite :

- **Sécurité / autorisation** : les routes d'historique et de gestion de compte ne vérifient pas encore la propriété des ressources côté serveur (l'identité est gérée principalement côté client). Un contrôle d'autorisation serveur (vérification de session/jeton) est à renforcer.
- **Paramètres financiers** : le capital initial et le *stop-loss* sont saisis et stockés mais pas encore intégrés au calcul ; les frais de transaction et le *slippage* ne sont pas modélisés.
- **Indicateurs** : le RSI utilise une moyenne exponentielle (approche simplifiée vs. RSI de Wilder) ; MACD et MA sont toujours à en position.
- **Robustesse** : peu de gestion d'erreurs autour des appels réseau (yfinance) et de l'écriture en base.

8.3 Perspectives

Renforcement de l'autorisation serveur, prise en compte du stop-loss et des frais dans le moteur, ajout de stratégies et de métriques, sauvegarde de stratégies et listes de suivi (tables déjà modélisées), et amélioration de la robustesse réseau.

9 Manuel utilisateur

9.1 Prérequis

Python 3.11+, Node.js 20+ et npm 10+.

9.2 Installation et lancement

Backend.

```
1 cd backend
2 python -m venv .venv
3 .\.venv\Scripts\Activate.ps1      # Windows (PowerShell)
4 # source .venv/bin/activate      # Linux / macOS
5 pip install -r requirements.txt
6 uvicorn main:app --reload --host 127.0.0.1 --port 8000
```

API disponible sur <http://127.0.0.1:8000>, documentation interactive sur <http://127.0.0.1:8000/docs>.

Frontend.

```
1 cd frontend
2 npm install
3 npm start
```

Application disponible sur <http://localhost:4200>.

9.3 Utilisation pas à pas



FIGURE 8 – Page de connexion : authentification Google ou accès en mode démo.

1. **Connexion** : sur la page de connexion, se connecter avec Google ou cliquer sur *Mode démo* *z* pour utiliser l'application sans compte.
2. **Données de marché** (page *Data*) : choisir un indice et une période pour afficher le graphique des cours.
3. **Backtest** (page *Backtests*) : sélectionner indice, période, stratégie et ajuster les paramètres, puis cliquer sur *Lancer z*.
4. **Résultats** (page *Results*) : consulter les métriques (rendement, Sharpe, drawdown, trades, taux de réussite), la courbe d'équité et le tableau des signaux ; naviguer dans l'historique récent.
5. **Tableau de bord** (page *Dashboard*) : vue synthétique des backtests et du meilleur rendement.
6. **Paramètres** (page *Settings*) : modifier le nom d'utilisateur, ajuster les préférences d'interface, supprimer le compte.

9.4 Exécution des tests (développeurs)

```
1 # Backend
2 cd backend
3 pip install -r requirements-dev.txt
```

```

4  pytest                                # 58 tests
5
6  # Frontend
7  cd frontend
8  npm install
9  npm test                             # ou : ng test --no-watch (41 tests)

```

10 Conclusion

Ce projet nous a permis de concevoir et de livrer une plateforme web de backtesting fonctionnelle, dépassant les objectifs du MVP grâce à l'intégration de données temps réel, d'une base de données, de métriques avancées et d'une authentification. La démarche Agile, le découpage en pôles et l'outillage collaboratif ont structuré un développement incrémental. La campagne de tests (99 tests automatisés, tous au vert) garantit la fiabilité du cœur métier et facilite la maintenance future. Les limitations identifiées constituent une feuille de route claire pour la poursuite du projet.

11 Références

- Angular — <https://angular.dev>
- Angular Material — <https://material.angular.io>
- lightweight-charts (TradingView) — <https://tradingview.github.io/lightweight-charts/>
- FastAPI — <https://fastapi.tiangolo.com/>
- Pydantic — <https://docs.pydantic.dev/>
- Pandas — <https://pandas.pydata.org/docs/>
- yfinance — <https://github.com/ranaroussi/yfinance>
- SQLAlchemy — <https://docs.sqlalchemy.org/>
- Authlib — <https://docs.authlib.org/>
- pytest — <https://docs.pytest.org/>
- Vitest — <https://vitest.dev/>
- Investopedia — Backtesting — <https://www.investopedia.com/terms/b/backtesting.asp>
- Formation Git : D. Ranc, *Introduction à GIT pour TSP1A* z, PRO3600.
- Méthode Agile : H. Charbonnier, *PRO3100* z GATE z.

12 Lexique

API

Interface de programmation permettant au frontend de communiquer avec le backend (échanges en JSON).

Backtesting

Simulation d'une stratégie d'investissement sur des données historiques pour en évaluer la viabilité.

OHLCV

Format standard de données financières : Open, High, Low, Close, Volume.

Drawdown (max)

Pire perte depuis un sommet de la courbe d'équité.

Ratio de Sharpe

Rendement ajusté du risque (rendement moyen rapporté à la volatilité), ici annualisé.

MACD

Moving Average Convergence Divergence : indicateur fondé sur la différence de deux moyennes exponentielles.

RSI

Relative Strength Index : oscillateur de momentum borné entre 0 et 100.

MA Crossover

Stratégie de croisement de moyennes mobiles (rapide/lente).

Signal

Position décidée par la stratégie : +1 (long), -1 (court), 0 (neutre).

Courbe d'équité

Évolution de la valeur cumulée d'un portefeuille au cours du temps.

Look-ahead bias

Biais consistant à utiliser une information future ; évité ici par le décalage du signal.

OAuth

Protocole d'autorisation déléguée (ici via Google).

ORM

Object-Relational Mapping : correspondance objets/tables (SQLAlchemy).

SPA

Single Page Application : application web ne rechargeant qu'une seule page.

MVP

Produit Minimum Viable : version minimale validant le concept.

Sprint

Cycle de développement court et itératif (méthode Agile).